

1. Basic Concurrency: Worker Pool

โจทย์: จงเขียนฟังก์ชันชื่อ RunWorkers โดยรับ parameter 2 ตัวคือ numWorkers (จำนวนคนงาน) และ numJobs (จำนวนงานทั้งหมด)

ให้สร้าง Workers จำนวน numWorkers ตัว ที่ทำงานพร้อมๆ กัน (Concurrent)

แต่ละ Worker จะคอยรับงานจาก Channel แล้วพิมพ์ข้อความว่า "Worker [ID] processing job [Job ID]"

ให้จำลองการทำงานด้วย time.Sleep เล็กน้อย (เช่น 1 วินาที)

Main function ต้องรอให้ทุกงานถูกทำจนเสร็จสิ้นจริงๆ ก่อนถึงจะจบโปรแกรม

2. Thread-Safety: Safe Counter

โจทย์: จงสร้าง struct ชื่อ SafeCounter ที่ภายในเก็บค่า count (int) และมี Methods 2 ตัวดังนี้:

Inc(): สำหรับเพิ่มค่า count ทีละ 1

Value(): สำหรับคืนค่า count ปัจจุบันออกมา

เงื่อนไข: โค้ดนี้ต้องรองรับกรณีที่มี Goroutines จำนวนมาก (เช่น 1,000 routines) เรียกใช้ Inc() พร้อมกัน โดยที่ค่าสุดท้ายต้องถูกต้องแม่นยำ ห้ามเกิด Race Condition

3. Interfaces: ระบบคำนวณพื้นที่ (Shape)

โจทย์:

ให้ประกาศ (Define) Interface ชื่อ Shape ที่มี method ชื่อ Area() float64

สร้าง Struct 2 ตัวชื่อ Rectangle (มี field Width, Height) และ Circle (มี field Radius)

Implement method Area ให้กับ Struct ทั้งสองเพื่อคำนวณพื้นที่

เขียนฟังก์ชันแยกออกมา 1 ตัว ชื่อ PrintArea(s Shape) ที่รับ Shape รูปทรงไหนก็ได้ แล้วพิมพ์ขนาดพื้นที่ออกมา

4. Logic & Map: หาคู่ตัวเลข (Two Sum)

โจทย์: กำหนดให้มี Slice ของตัวเลข nums := []int{2, 7, 11, 15} และค่าเป้าหมาย target := 9

จงเขียนฟังก์ชันที่รับ nums และ target แล้วคืนค่า index ของตัวเลข 2 ตัวใน slice ที่บวกกันแล้วได้เท่ากับ target พอดี

สมมติว่ามีคำตอบที่ถูกต้องแน่นอนเพียง 1 คู่

เงื่อนไข: ห้ามใช้ Loop ซ้อน Loop (Double for-loop) ต้องใช้วิธีที่มีประสิทธิภาพ Time Complexity ดีกว่า $O(n^2)$

5. HTTP Handler: สร้าง JSON API ง่ายๆ

โจทย์: จงใช้ package net/http เขียน Web Server ที่รันบน port 8080 และมี endpoint ชื่อ /hello โดยมีเงื่อนไขดังนี้:

ต้องรับ Request Method เป็น POST เท่านั้น

รับ Body เป็น JSON: {"name": "Somchai"}

แกะค่า name ออกมาแล้วตอบกลับ (Response) เป็น JSON: {"message": "Hello Somchai"}

กรณีที่ Method ไม่ใช่ POST หรือส่ง JSON มาผิด format ให้ return HTTP Error Code ที่เหมาะสมกลับไป